Cyprus University of Technology

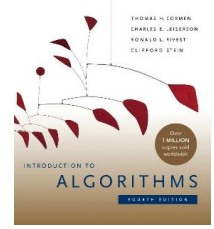
CSE 226
Algorithms and Complexity

Final Exam Review

Herodotos Herodotou

Book

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. ***Introduction to Algorithms***. The MIT Press; 4th Edition, 2022, ISBN: 9780262046305.
- *Ελληνική μετάφραση*: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. ***Εισαγωγή στους Αλγόριθμους***, Έκδοση 3, Πανεπιστημιακές Εκδόσεις Κρήτης, 2016, ISBN: 960-524-225-7.
- *Online Edition*: http://library.cut.ac.cy/

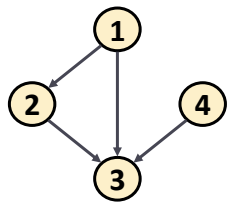


Final Exam Material

Α/Α	Διάλεξη	Κεφάλαιο
10	Elementary Graph Algorithms – Στοιχειώδεις Αλγόριθμοι Γραφημάτων (Part 1)	20.1, 20.2
11	Elementary Graph Algorithms – Στοιχειώδεις Αλγόριθμοι Γραφημάτων (Part 2)	20.3, 20.4
12	Minimum Spanning Trees – Ελαφρύτατα Συνδετικά Δένδρα (Part 1)	21.0, 21.1, 21.2
13	Minimum Spanning Trees – Ελαφρύτατα Συνδετικά Δένδρα (Part 2)	21.2, 19.1, 19.2
14	Single-Source Shortest Paths – Ομοαφετηριακές Ελαφρύτατες Διαδρομές (Part 1)	22.0, 22.1
15	Single-Source Shortest Paths – Ομοαφετηριακές Ελαφρύτατες Διαδρομές (Part 2)	22.2, 22.3
16	All-Pairs Shortest Paths – Πανζευκτικές Ελαφρύτατες Διαδρομές (Part 1)	23.0, 23.1
17	All-Pairs Shortest Paths – Πανζευκτικές Ελαφρύτατες Διαδρομές (Part 2)	23.2, 23.3
18	Maximum Flow – Μέγιστη Ροή (Part 1)	24.0, 24.1, 24.2
19	Maximum Flow – Μέγιστη Ροή (Part 2)	24.2, 24.3
20	NP-Completeness – NP-Πληρότητα (Part 1)	34.0, 34.1
21	NP-Completeness – NP-Πληρότητα (Part 2)	34.1, 34.2

Graph Representation

- **Adjacency lists** (**κατάλογοι γειτνίασης**)
- **Adjacency matrix** (**πίνακας γειτνίασης**)
- Tradeoffs:
  - Running Time Vs Storage Complexity
  - What makes a graph **dense** (**πυκνό γράφημα**)?
  - What makes a graph **sparse** (**αραιό γράφημα**)?
  - What about **complete graphs** (**πλήρες γράφημα**)?



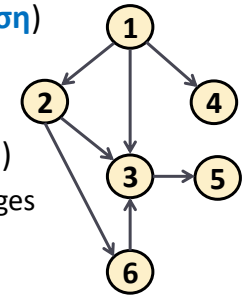

```
Adjacency Lists:
Adj[1] = {2,3}
Adj[2] = {3}
Adj[3] = {}
Adj[4] = {3}
```

Adjacency Matrix:

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

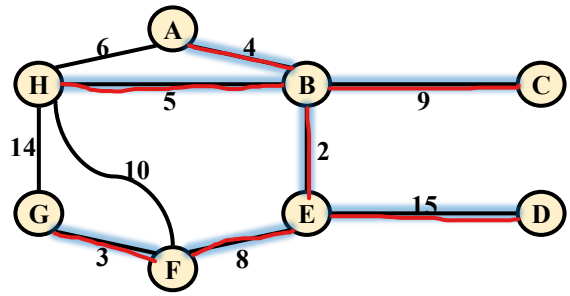
Basic Graph Algorithms

- **Breadth-first Search** (**Οριζόντια Διερεύνηση**)
  - What can we use BFS for?
  - What is the running time?
- **Depth-first Search** (**Καθοδική Διερεύνηση**)
  - Tree edges, back edges, cross and forward edges
  - What can we use DFS for?
  - What is the running time?
- **Topological Sort** (**Τοπολογική Ταξινόμηση**)
  - Examples: getting dressed, project dependency
  - To what kind of graph does topological sort apply?
  - What is the running time?



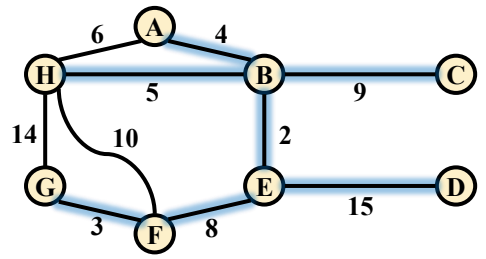
Minimum Spanning Tree

- **Spanning tree** (**συνδετικό δέντρο**)
  - How many edges does it have? $V-1=E$
- Minimum spanning tree (**Ελαφρύτατο Συνδετικό Δένδρο**) MST
  - **Optimal substructure** (**βέλτιστη υποδομή**)
  - **Min-weight edge theorem** (enables greedy approach)



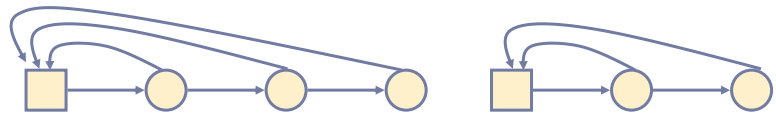
Minimum Spanning Tree Algorithms

- **Prim's algorithm**
  - What is the bottleneck in Prim's algorithm?
  - What is the running time?
- **Kruskal's algorithm**
  - What is the bottleneck in Kruskal's algorithm?
  - What is the running time?



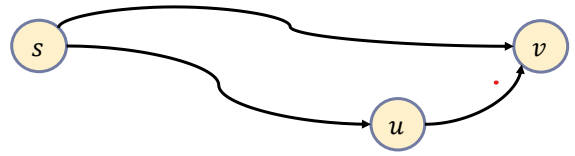
Disjoint-Set Union

- We can represent disjoint sets as linked lists, every element stores pointer to list head
- What is the cost of merging two sets A and B?
- What is the maximum cost of merging n 1-element sets into a single n-element set?
- How did we improve this? By how much?



Single-Source Shortest Path

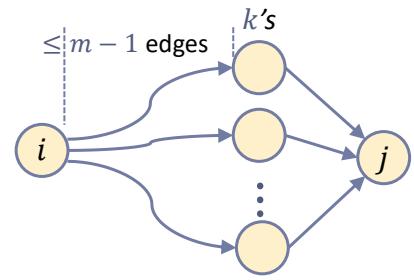
- **Optimal substructure** (**βέλτιστη υποδομή**)
- Key idea: **Relaxation** (**Χαλάρωση**)



- What does the **Bellman-Ford algorithm** do?
  - What is the running time?
- What does **Dijkstra's algorithm** do?
  - When does Dijkstra's algorithm not apply?
  - What is the running time?

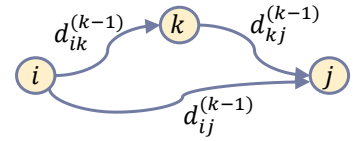
All-Pairs Shortest Path

- **Matrix Multiplication Approach**
  - $l_{ij}^{(m)}$ = weight of a shortest path from i to j that uses $\leq m$ edges, $l_{ij}^{(m)} = \min_{1 \leq k \leq n} \left\{ l_{ik}^{(m-1)} + w_{kj} \right\}$
  - Optimization: **repeated squarings** (**επαναληπτικοί τετραγωνισμοί**)
  - What is the running time?



All-Pairs Shortest Path

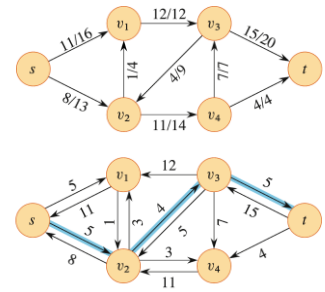
- **Floyd-Warshall Algorithm**
  - $d_{ij}^{(k)}$ = weight of a shortest path from i to j with intermediate vertices belonging to the set $\{1,2,\ldots,k\}$
  - $d_{ij}^{(k)} = \min\left\{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right\}$
  - What is the running time?



- **Johnson's Algorithm**
  - Key idea: reweight each edge $(u, v) \in E$ by $w_h(u, v) = w(u, v) + h(u) - h(v)$ to eliminate negative weights
  - Run Bellman-Ford to calculate the function h
  - Run Dijkstra's for each vertex
  - What is the running time?

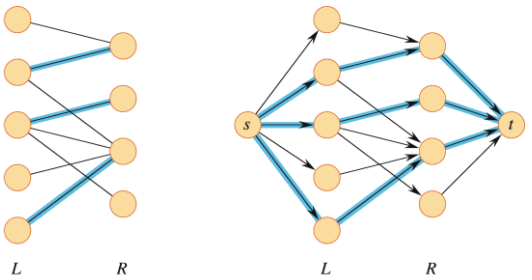
Maximum Flow

- What is a **flow network** (**δίκτυο ροής**)?
- What are the main 2 properties of **flow** (**ροή**)?
- Important notions in **Ford-Fulkerson Method**
  - **Residual network** (**Υπολειπόμενο δίκτυο**)
  - **Augmenting path** (**Αυξητική διαδρομή**)
  - **Cut** (**Τομή**)
- **Max-flow Min-cut Theorem**
- **Edmonds-Karp algorithm**
  - What is the running time?



Maximum Bipartite Matching

- What is a **bipartite graph** (**διμερής γράφος**)?
- What is **matching** (**αντιστοίχιση**)?
- What is **maximum matching**?
  - How do we solve it?
  - What is the running time?



NP-Completeness

1. What do we mean when we say a problem is in **P**?
2. What do we mean when we say a problem is in **NP**?
3. What is the relation between **P** and **NP**?
4. What, intuitively, does it mean if we can **reduce** problem P to problem Q?
5. How do we reduce P to Q?
6. What does it mean if Q is **NP-Complete**?
7. What does it mean if Q is **NP-Hard**?

